

JSON

JSON is the "gold standard" for data exchange because it is **Unicode-based** (supports all languages), **Human-readable**, and **Compact** enough to be efficient without becoming unreadable gibberish.

1. Key Properties of JSON

- **Recursive Structure:** You can nest data as deeply as you need. A struct can contain another struct, which contains an array, which contains integers. JSON handles this hierarchy perfectly.
- **Human-Readable:** Unlike binary formats, a human can open a JSON file and understand the data relationships.
- **Unicode:** Everything is represented as Unicode characters, making it globally compatible.

2. JSON Marshalling

"**Marshalling**" is the process of taking a Go object (like a Struct) and generating a JSON representation of it.

- **The Function:** `json.Marshal(object)`
- **The Result:** It returns two things:
 1. **Byte Array ([]byte):** This contains the Unicode/JSON data.
 2. **Error:** If something goes wrong, this tells you why. If it works, it is nil.

```
type Person struct {
    Name    string
    Address string
    Phone   string
}

p1 := Person{Name: "Joe", Address: "A St.", Phone: "123"}

// Marshalling p1 into a JSON byte array
barr, err := json.Marshal(p1)

if err == nil {
    fmt.Println(string(barr)) // Converts byte array to string for reading
}
```

3. JSON Unmarshalling

"**Unmarshalling**" is the reverse: taking a JSON byte array and unpacking that data into a Go Struct.

- **The Function:** `json.Unmarshal(data, &target)`
- **The Arguments:**
 1. The byte array (barr) containing the JSON.

2. The **address** of the variable where you want to save the data (use the & symbol).

- **The "Fit" Rule:** The Go struct must have field names that match the JSON attributes for the data to land in the right spot.

```
var p2 Person           // Create an empty struct first
err := json.Unmarshal(barr, &p2) // Fill p2 with data from barr

if err == nil {
    fmt.Println(p2.Name) // Joe
}
```

4. Logic & Error Handling

- **The Byte Array:** Go treats JSON as a []byte (a slice of bytes) because JSON is essentially a string of characters.
- **Verification:** Always check the err variable. If your Go struct doesn't "fit" the JSON (e.g., the JSON has a field called "Email" but your struct doesn't), Unmarshal might throw an error or simply skip that field.

Expert Takeaway:

You will use **json.Unmarshal** every time you receive data from a frontend or a third-party API.

Pro Tip: In the real world, **JSON** keys are usually lowercase (e.g., "name": "Joe"), while Go fields must be Uppercase to be **exported** (e.g., Name). We bridge this gap using **Struct Tags**.

Example: Name string json:"name" tells Go: "In code, I call this Name, but in JSON, it is called name."